RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

Machine Learning (MLOps) Tutorial

Urban Air Quality Intelligence System with Predictive Analytics

SDG 11 : Sustainable societies and communities

Team Number: **17**

USN	**Name**
1RV23AI132	Vineet Raj
1RV23AI135	Abhinav Potharaju
1RV24AI407	Mahantesh B P
1RV24AI406	L Moryakantha

Agenda

- Introduction
- Motivation (Why this title)
- Problem Definition
- Objectives
- Data set Description
- Risk Analysis
- Tools and Techniques used
- Phase I : Machine Learning Model and its analysis
- Phase II : Building MLOPs

Introduction

- Rapid urbanization has increased the air pollution, making **PM2.5 prediction critical** for public health and policy decisions.
- Traditional ML models often fail in production due to **data drift, poor monitoring, and lack of automation**.
- **MLOps bridges this gap** by integrating ML development with deployment, monitoring, and governance practices.
- This project designs an **end-to-end MLOps pipeline** for urban PM2.5 prediction using real-world air quality data.
- It combines **model training, CI/CD-ready deployment, monitoring, and ethical governance** in a unified system as **MLOps pipeline.**

Motivation (Why this title)

- **Severe Urban Pollution Problem :-** Rapid urbanization has led to dangerously high PM2.5 levels.
- **Traditional Models Are Not Deployment-Ready:-** Most air quality models are not fully functional (fails,etc.)
- **Lack of End-to-End Automation :-** Manual or without automation cause delays and inconsistencies.
- **Model Drift & Reliability Issues:-** Air quality patterns change frequently.
- **Need for Continuous Monitoring & Governance:-** Real-world systems require performance tracking, fairness checks, and auditability.
- **Bridging the Research–Production Gap:-** More focus on algorithms, but lack CI/CD, monitoring, and operational robustness.
- **MLOps Enables Scalable & Sustainable AI:-** Integrating MLOps ensures automation, reproducibility, transparency, and long-term reliability.

Problem Definition

- Urban areas face **frequent and unpredictable PM2.5 spikes**.
- Traditional air quality systems are **reactive.**
- Existing ML models focus only on **accuracy**, ignoring deployment and monitoring.
- **Data drift, sensor noise, and model decay** reduce long-term reliability.
- Lack of **end-to-end MLOps pipelines (from dev to prod)** for air quality prediction.
- Difficulty in detecting **unusual or hazardous AQI conditions in real time.**

Objectives

- To **predict urban PM2.5 air quality levels** using 3 machine learning models(Linear Regression, Random Forest, XGBoost).
- To **compare 3 ML models** and select the best-performing one.
- To **build an end-to-end MLOps pipeline** which **deploy the trained model as a scalable API ,enable experiment tracking and model versioning, monitor model performance , data drift and system health and ensure governance, fairness, and transparency**

Data-set Description

Dataset 1: Time-Series Air Quality Data of India (2010–2023)

Source: Kaggle

Link: Dataset 2

Description:

- This dataset contains **station-level time-series air quality data** collected across different locations in India from **2010 to 2023**.
- Each CSV file represents measurements from a specific monitoring station.
- The data is recorded at regular time intervals.
- The data has been collected from the **Central Control Room for Air Quality Management**, which indicates that it is a reliable and authoritative source.

Key Features Used:

- Timestamp (Date & Time of measurement)
- PM2.5
- PM10
- O3 (Ozone)
- CO (Carbon Monoxide)

Use in Project:

- This dataset provides **long-term historical pollution trends** and high-frequency measurements.

Data-set Description

Dataset 2: Indian Cities AQI Dataset (2020–2024)

Source: Kaggle

Link: Dataset 2

Description:

- This dataset contains **city-level air quality data** for major Indian cities from **2020 to 2024**.
- It represents aggregated pollution data at the city level rather than individual stations.
- The data has been collected from the Central Pollution Control Board (CPCB)'s repository - CPCB

Key Features Used:

- Timestamp
- PM2.5
- PM10
- O3
- CO

Use in Project:

- This dataset adds **geographical and urban-level pollution patterns**.

Data-set Description

Dataset Integration & Final Dataset

- Both datasets were **cleaned, standardized, and merged** based on **common pollutant columns and timestamp**.
- Column names were normalized (e.g., different date/time formats unified into a single Timestamp column).
- Missing values were handled using **median imputation**.

Final Combined Dataset:

master_airquality_clean.csv

Final Columns Used for Modeling:

- Timestamp
- PM2.5 (Target variable)
- PM10
- O3
- CO
- hour, dayofweek, month (engineered time features)
- Source

Risk Analysis

There are 10 risks which we had discovered and assessed them with score out of 25 (5x5) in which first metric is **Probability** and then second is **Impact** Assessment.

Notations : Rx (Risk x : x belong to 1 -10)

R1. Incomplete or Poor-Quality Environmental Data. (4/5 x 4/5 =16/25)

R2. Model Underperformance due to data drift and bias.(3/5 x 4/5 =12/25)

R3. Integration Issues among MLOps Tools.(3/5 x 3/5 =9/25)

R4. Model Reproducibility failure due to missing environment versions.(2/5 x4/5 =8/25)

R5. Container deployment failure or API downtime.(3/5 x 3/5 =9/25)

R6. Misinterpretation of Air Quality Forecasts (Stakeholder misunderstanding).(2/5 x 4/5 =10/25)

R7. Loss of model or data versions due to repository issues.(1/5 x 4/5 =4/25)

R8. Lack of Model Generalization to New Cities or Unseen Locations.(4/5 x 3/5 =12/25)

R9. Violating Data Usage Rights (Non-open Datasets).(1/5 x 5/5 =5/25)

R10. Inadequate monitoring or failure to detect Performance Degradation.(3/5 x 4/5 =12/25)

Tools and Techniques used

Tools

- **Python** – Core programming language for data processing and modeling
- **Pandas & NumPy** – Data cleaning, preprocessing, and analysis
- **Scikit-learn** – Model training, evaluation, and preprocessing pipelines
- **XGBoost** – High-performance regression model for PM2.5 prediction
- **FastAPI** – REST API for real-time model inference
- **Docker & Docker Compose** – Containerized deployment and environment consistency
- **Prometheus** – Monitoring API performance and system metrics
- **Grafana** – Dashboard
- **Weights & Biases (W&B) , CometML** – Experiment tracking, model versioning, and logging
- **AIF360** – Fairness, bias, and governance analysis
- **KubeFlow**– for pipeline orchestration

Techniques

- Data cleaning, normalization, and feature engineering
- Time-based feature extraction (hour, day, month)
- Regression modeling and model comparison
- Hyperparameter tuning and performance evaluation (RMSE, R^2, MAE)
- Anomaly detection using residual analysis
- Model explainability and governance practices
- Containerized deployment and monitoring (MLOps lifecycle)

Phase I – ML Life Cycle

Exploratory Data Analysis

- Analyzed historical air quality data to understand pollution trends and patterns
- Examined distribution of PM2.5 and other pollutants across time and locations
- Identified missing values, outliers, and inconsistencies in sensor readings
- Studied temporal patterns (hourly, daily, monthly variations in pollution)
- Analyzed correlations between PM2.5 and pollutants like PM10, CO, O_3
- Compared pollution levels across cities and industrial monitoring stations
- Visualized data using SHAP summary plots for interpretability, residual plots and histograms for error analysis, and bar charts for model performance comparison.
- Insights from EDA guided feature engineering and model selection

Phase I – ML Life Cycle

List of algorithms chosen to build ML model

Linear Regression

- Used as a **baseline model**
- Helps understand linear relationships between pollutants and PM2.5

Random Forest Regressor

- Handles **non-linear relationships**
- Robust to noise and overfitting
- Performs well on tabular environmental data

XGBoost Regressor

- Gradient boosting–based ensemble model
- Captures complex interactions between features
- Provides **high accuracy and generalization**
- Selected as the **final best-performing model**

Phase I – ML Life Cycle

Metrics used to evaluate the ML model

RMSE (Root Mean Squared Error)

- Primary evaluation metric
- Penalizes large prediction errors
- Suitable for continuous PM2.5 prediction

R^2 Score (Coefficient of Determination)

- Measures how well the model explains variance in PM2.5
- Helps compare model performance

Model-wise RMSE Comparison

- Linear Regression – baseline performance
- Random Forest – captures non-linear patterns
- XGBoost – best generalization with lowest RMSE

Phase I – ML Life Cycle

Evaluation of ML model

1. Models evaluated on **unseen test data** to ensure generalization
2. **RMSE (Root Mean Square Error)** used as primary metric
   a. Penalizes large prediction errors
   b. Suitable for continuous PM2.5 prediction
3. **Linear Regression**
   a. Baseline model
   b. Higher error due to linear assumptions
4. **Random Forest**
   a. Captures non-linear relationships
   b. Better performance than Linear Regression
5. **XGBoost**
   a. Lowest RMSE among all models
   b. Handles complex patterns and feature interactions efficiently
6. **Best model selected based on minimum RMSE**
7. Model performance logged and compared using **Weights & Biases**
8. Final best model saved and versioned for deployment

Phase I – ML Life Cycle

Identification of Best Model

- Three models were trained: **Linear Regression, Random Forest, and XGBoost.**
- All models were evaluated using **RMSE (Root Mean Square Error).**
- **Lower RMSE means better prediction accuracy.**
- Linear Regression showed **high error** and could not capture complex patterns
- Random Forest performed better but showed **slight overfitting**
- XGBoost achieved the **lowest RMSE among all models**
- XGBoost handled **non-linear relationships and feature interactions** well
- It showed **stable performance on unseen test data**
- **XGBoost was selected as the final and best-performing model**

Phase II – MLOPs Life Cycle

	Component	MLOPs Tools
1	Data Management	Grafana ,Git (versioning)
2	Experiment Tracking and Version Control	Weights & Biases (W&B) , Git ,CometML
3	Automation and Pipeline Orchestration	Kubeflow , Docker
4	Model Deployment & Serving	Docker , Fast+API ,Github Actions
5	Monitoring & Governance and collaboration	AIFairness 360 , Prometheus

Phase II – MLOPs Life Cycle

Data Management

Data Collection

- Air quality data collected from **cities and monitoring stations from CPCB** (CSV files).
- Data includes PM2.5, PM10, gases, weather, and timestamps.

Data Cleaning (Using Pandas)

- Handle missing values (mean/median imputation).
- Remove duplicate rows and invalid sensor readings.
- Convert timestamps into usable time features (hour, day, month).

Data Transformation

- Feature scaling (normalization/standardization).
- Feature engineering like lag values and rolling averages.

Data Versioning (Using Git)

- Raw, cleaned, and processed datasets are tracked in Git.
- Ensures reproducibility and rollback if data changes.

Visualization Support

- Grafana dashboards can visualize stored metrics (later stage).

Phase II – MLOPs Life Cycle

Experiment Tracking and Version Control

Initialize Experiment Tracking

- Login to W&B and initialize a run before model training.

Log Hyperparameters

- Learning rate, depth, number of trees, etc. are logged.

Train Multiple Models

- Linear Regression
- Random Forest
- XGBoost

Log Metrics

- RMSE values for each model logged automatically.
- Comparison between models done visually in W&B dashboard.

Model Versioning

- Best model saved as a W&B artifact.
- Enables rollback to older models if needed.

Code Version Control

- Git tracks training scripts and notebooks.

Phase II – MLOPs Life Cycle

Automation & Pipeline Orchestration

Containerization (Docker)

- Training and inference environments packed into Docker images.
- Ensures consistency across systems.

Pipeline Definition (Kubeflow)

- Define pipeline steps:
  - Data preprocessing
  - Model training
  - Evaluation
  - Model storage

Pipeline Execution

- Kubeflow executes steps automatically.
- Handles dependencies between steps.

Scalability

- Pipelines can run on multiple nodes for large datasets.

Phase II – MLOPs Life Cycle

Model Deployment & Serving

Model Export

- Best model saved as .pkl or .joblib.

API Creation (FastAPI)

- /predict endpoint created for PM2.5 prediction.
- Input features passed as JSON.

Model Serving

- Model loaded inside FastAPI application.
- Predictions returned in real time.

Application Server

- Uvicorn used to run the FastAPI app.

Containerized Deployment

- API wrapped in Docker container.

CI/CD (GitHub Actions)

- On code push:
  - Build Docker image
  - Run tests
  - Deploy updated API automatically

Phase II – MLOPs Life Cycle

Monitoring, Governance & Collaboration

API Monitoring (Prometheus)

- Tracks request count, latency, and errors.
- /metrics endpoint exposed in FastAPI.

Model Performance Monitoring

- Monitor prediction distribution over time.
- Detect sudden changes (concept drift).

Fairness & Bias Analysis (AIF360)

- Check if predictions are biased across regions or time.
- Generate fairness metrics and reports.

Governance

- Maintain:
  - Model cards
  - Data cards
  - Audit logs

Collaboration

- Teams collaborate using:
  - Git for code
  - W&B dashboards for experiments
  - Shared monitoring dashboards